

Skriptbeschreibungen:

Für CONFIG, MASTER.CLI, 0_PHASE_DETECTION.PY,
1_VIDEO_ANALYSIS_PTG.PY, 1_VIDEO_ANALYSIS.PY,
2_OFFLINE_CALIBRATION.PY, 2_OFFLINE_CALIBRATION_PTG.PY,
3_EYETRACKER_LOAD.PY, 4_SYNCHRONIZATION.PY,
5_CALIBRATION_APPLY.PY, 5_CALIBRATION_APPLY_PTG.PY und
6_COMPARISON.PY siehe Masterarbeit

[utils] Hilfsskripte für MASTER.CLI:

ADAPTIVE_SYNC_DETECTION.PY. Dieses Skript implementiert eine Multi-Method-Detektion für extrem schwache Audio-Marker (funktioniert bei SNR < -50 dB). Es kombiniert drei Verfahren: Template Matching (Cross-Correlation mit synthetischem Referenz-Ton), den Goertzel-Algorithmus (effiziente Einzelfrequenz-Analyse) und STFT Peak Detection (Frequenz-Zeit-Analyse). Ein Consensus-Mechanismus wertet die Ergebnisse aller Methoden aus und wählt die zuverlässigsten Kandidaten. Es wird von debug_0_phase_detection.py als Fallback verwendet, wenn Standard-Ton-Detektion versagt.

ANALYSIS_PIPELINE.PY. Dieses Skript orchestriert die komplette Pipeline-Ausführung für eine Versuchsperson. Es führt alle debug_*.py Skripte in der korrekten Reihenfolge aus (debug_0 → debug_1 → offline_calibration → debug_3 → debug_4 → debug_5 → debug_6), verwaltet Environment-Variablen für die Skript-Kommunikation, implementiert intelligentes Cache-Management (erkennt kompatible Zwischenergebnisse aus früheren Runs und kopiert diese statt neu zu berechnen), und unterstützt beide Pipelines (MediaPipe und ptgaze) parallel. Es wird von master_cli.py für die Batch-Verarbeitung mehrerer VPs verwendet.

VP_DATA_MANAGER.PY. Dieses Skript implementiert die automatische Datei-Erkennung für Versuchspersonen. Es scannt den Basis-Ordner nach VP-Ordern (4-Zeichen-Codes), erkennt automatisch alle relevanten Dateien (ASC-Dateien, experiment_sync_log.json, Videos mit 60Hz/25Hz, Calibration-JSONs), validiert die Vollständigkeit der Daten, und verwaltet existierende Analyse-Runs. Zusätzlich implementiert es das Cache-Kriterien-System (CACHE_CRITERIA), das definiert welche Config-Felder für jeden Pipeline-Schritt relevant sind (z.B. debug_0 braucht nur video_fps-Match, offline_calibration braucht video_fps + calib_mode).

CALIBRATION_MODE_MANAGER.PY. Dieses Skript definiert und verwaltet verschiedene Kalibrierungs-Strategien für die Offline-Kalibrierung. Es bietet vordefinierte Modi wie FullCalib (alle Punkte), BegEnd (ohne Mid-Kalibrierung), OnlyBeg (nur Anfangs-Kalibrierung), BegFirst10EndFix (reduzierte Kalibrierung mit Fixationen) und weitere Varianten. Bei Anwendung eines Modus filtert es die Kalibrierungspunkte aus phases_detected.json entsprechend der Strategie und erstellt eine modifizierte JSON-Datei, die von offline_calibration.py verwendet wird. Dies ermöglicht systematische Experimente zur Frage, wie viele Kalibrierungspunkte für gute Genauigkeit benötigt werden.

[statistic] Vorbereitung statistische Analyse:

DATA_AVAILABILITY_SCANNER.PY. Dieses Skript scannt alle VP-Ordner nach vorhandenen Analysedaten und prüft deren Vollständigkeit pro Konfiguration (video_fps × calib_mode). Es identifiziert welche VP-Config-Kombinationen vollständig analysiert wurden (MediaPipe + ptgaze + EyeLink vorhanden), welche teilweise vorliegen und welche fehlen. Zusätzlich berechnet es automatisch die Machbarkeit der Forschungsfragen (FF1–FF5) basierend auf der Datenverfügbarkeit und generiert einen strukturierten Report (JSON-Export möglich). Es wird vom Statistikmodus verwendet, um dem Benutzer eine Übersicht zu geben und fehlende Daten zu identifizieren.

STATISTICAL_DATA_PREP.PY. Dieses Skript bereitet die Rohdaten aus der Pipeline für die statistische Analyse in R auf. Es lädt die drei Datenquellen (MediaPipe, ptgaze, EyeLink), konvertiert EyeLink-Pixelkoordinaten zu Grad (Sehwinkel), downsamplet EyeLink von 2000 Hz auf Video-Framerate, berechnet die Lag-Korrektur via Cross-Correlation (CV-Daten sind typischerweise 100–300 ms verzögert) und merged alle Quellen auf Frame-Level. Es setzt Ausschluss-Flags (Blinks, Outside Monitor) ohne sie anzuwenden – die finale Ausschlussentscheidung erfolgt in R für maximale Transparenz und Reproduzierbarkeit.

STATISTICAL_PIPELINE.PY. Dieses Skript orchestriert den gesamten Workflow zur Erstellung des Analysedatensatzes. Es koordiniert den DataAvailabilityScanner (was ist vorhanden?), kann fehlende Daten mit der bestehenden Pipeline nachgenerieren, ruft StatisticalDataPrep für jede VP/Config auf, und aggregiert die Ergebnisse VP-übergreifend. Ein zentraler EyeLink-Cache vermeidet redundantes Laden und Konvertieren der 2000-Hz-Daten. Das Skript filtert nur erlaubte Kalibrierungskonfigurationen (K1=FullCalib, K4=BegFirst10EndFix) und exportiert zwei CSV-Dateien: Frame-Level (für LMM in R) und Trial-Level (Aggregate), plus Dokumentations-JSONs für Lag-Korrektur und Verarbeitungsprotokoll.

STATISTICAL_CLI.PY. Dieses Skript stellt das interaktive Terminal für den Statistikmodus bereit (Hauptmenü Option 2). Es zeigt eine Schnellübersicht der Datenverfügbarkeit, führt den Benutzer durch die Konfigurationsoptionen (Framerates, Kalibrierungen), ermöglicht selektive Nachgenerierung fehlender Daten, und bietet einen Preview-Modus für vorläufige Analysen mit nur verfügbaren Daten. Es ist die Benutzeroberfläche für die StatisticalPipeline-Klasse und wird von master_cli.py aufgerufen.

[shared] Geteilte Skripte für die Hauptanalyse:

SHARED_PUPIL_DETECTION.PY. Dieses Skript implementiert die vereinheitlichte Pupillen-Detektion für die MediaPipe-Pipeline. Es enthält zwei Hauptklassen: RobustPupilDetector für Frame-by-Frame-Extraktion der Pupillenpositionen mittels MediaPipe Face Mesh (Iris-Landmarks 468-477), und HeadPoseEstimator für 3D-Kopfhaltungs-Schätzung via cv2.solvePnP. Die Klasse implementiert eine 6-Ebenen-Verarbeitungspipeline: (1) Rohe Landmark-Extraktion, (2) Plausibilitätsprüfung (Augenabstand), (3) Qualitätsgewichteter Durchschnitt beider Augen, (4) IQR-basierte Outlier-Removal, (5) Temporale Glättung, (6) Median-Aggregation. Zusätzlich enthält es DetailedPupilDebugger für detaillierte CSV-Exports aller Verarbeitungsebenen. Wird von debug_1_video_analysis.py und offline_calibration.py verwendet.

SHARED_COORDINATE_UTILS.PY. Dieses Skript implementiert die zentrale Koordinaten-Transformation zwischen verschiedenen Systemen (Pixel ↔ Gradsehwinkel). Die Hauptklasse `CoordinateTransformer` konvertiert Bildschirm-Pixel zu Gradsehwinkeln basierend auf physikalischen Bildschirmparametern (Breite/Höhe in cm, Betrachtungsabstand) und umgekehrt. Zusätzlich enthält es `ScreenParameters` und `CalibrationMetadata` als Dummy-Klassen für Pickle-Kompatibilität mit älteren Kalibrierungsdateien, sowie Hilfsfunktionen für Outside-Monitor-Detection und Batch-Konvertierung von EyeLink-Daten. Wird von `debug_5` (Kalibrierung anwenden), `debug_6` (3-Wege-Vergleich) und `debug_7` (Interactive Timeline) verwendet.

SHARED_BLINK_DETECTION.PY. Dieses Skript implementiert die einheitliche Blink-Detektion für beide Pipelines (MediaPipe und ptgaze) basierend auf dem Eye Aspect Ratio (EAR) nach Soukupova & Cech (2016). Die Formel $EAR = (\|p_2 - p_6\| + \|p_3 - p_5\|) / (2 \cdot \|p_1 - p_4\|)$ berechnet das Verhältnis von Augenhöhe zu Augenbreite aus 6 MediaPipe-Landmarks pro Auge. Ein Blink wird erkannt, wenn der EAR-Wert für eine konfigurierbare Anzahl konsekutiver Frames unter einem Schwellwert (Standard: 0.15) liegt. Die Klasse `BlinkDetector` unterstützt drei Input-Formate: MediaPipe FaceLandmarks, NumPy-Arrays und ptgaze Face-Objekte.

SHARED_GAZE_DETECTION_PTGAZE.PY. Dieses Skript implementiert einen Wrapper um das ptgaze-Framework (ETH-XGaze CNN-Modell) für Gaze-Estimation. Die Klasse `PtgazeGazeDetector` liefert Blickwinkel direkt in Grad (pitch/yaw) statt Pupillenpositionen in Pixel. `PtgazeConfigGenerator` erstellt automatisch die benötigte OmegaConf-Konfiguration inkl. Kamera-Parameter aus der Video-Auflösung und lädt Pre-trained Model Weights automatisch herunter. Die Integration nutzt MediaPipe für Face Detection (konsistent mit der Haupt-Pipeline) und berechnet zusätzlich Head-Pose (Euler-Winkel) aus dem Face-Objekt. Wird von `debug_1_ptgaze.py` verwendet.

[extra] Weitere Hilfsskripte

BLINK_CHECK.PY. Dieses Skript annotiert Blinks in bereits verarbeiteten CSV-Dateien nachträglich mit einem optimierten Algorithmus. Es verwendet einen Hybrid-Ansatz aus Hidden Markov Model (HMM) und Peak-Detection auf dem Eye Aspect Ratio (EAR) Signal, kombiniert mit adaptiven Schwellenwerten (framerate-spezifisch für 25Hz vs. 60Hz). Validierungskriterien (minimale/maximale Blink-Dauer, Refractory Period, EAR-Percentil) reduzieren Falsch-Positive. Das Skript aktualisiert die Spalten `is_blink`, `eyes_closed` und `blink_count` in den `debug_5`-CSVs und erstellt optional Diagnose-Plots zur visuellen Inspektion. Es wird als Standalone-Tool nach der Hauptpipeline ausgeführt. Der EAR-MULTIPLIKATOR PRO FRAMERATE wurde manuell pro VP angepasst (siehe Skript).

VISUALIZE_GAZE_FROM_VIDEO.PY. Dieses Skript erstellt Visualisierungen der kalibrierten Blickbewegungen für den Kalibrierungstest (kly-VPs). Es extrahiert Pupillenpositionen bzw. Blickwinkel direkt aus Videos mittels MediaPipe und ptgaze, lädt die entsprechenden Kalibrierungsmodelle (PKL-Dateien), wendet die Kalibrierung an und plottet die resultierenden Bildschirmkoordinaten farbcodiert nach Zeit. Das Output sind 4 PNG-Seiten mit je 2 VPs × 2 Methoden nebeneinander, inklusive Kalibrierungspunkt-Overlay und gemeinsamer Colorbar. Es dient der qualitativen Bewertung der Kalibrierungsqualität für Smooth-Pursuit-Stimuli.

RUN_CALIBRATION_TEST_BATCH.PY. Dieses Skript führt automatisiert die Kalibrierungs-Pipeline (debug_0 + offline_calibration) für alle 8 Test-VPs (kly1–kly8) des Videokalibrierungstests aus. Es orchestriert beide Methoden (MediaPipe und ptgaze) sequenziell, extrahiert die R^2 -Werte aus den resultierenden PKL-Dateien und erstellt eine Vergleichstabelle mit Gruppenstatistiken (links_oben vs. rechts_unten Startrichtung). Die Ergebnisse werden als CSV exportiert.

EDFTOPIC.M. Dieses MATLAB-Skript visualisiert die mittels EyeLink 1000 erfassten Blickverläufe während der Videokalibrierung (kly1–kly8). Es lädt EDF-Dateien mittels der Edf2Mat-Toolbox, extrahiert die Blickkoordinaten (posX, posY) und plottet diese mit einem Blau-Rot-Farbgradienten zur zeitlichen Kodierung (turbo-Colormap). Die 10 Kalibrierungspunkte werden als transparente Kreise mit Nummerierung überlagert. Für jede VP wird die Zielgeschwindigkeit (200–500 px/s) und Startrichtung (links oben vs. rechts unten) im Titel angezeigt. Das Output sind 4 PNG-Dateien im DIN-A4-Format (300 dpi) mit je 2 VPs pro Seite. Das Skript dient der qualitativen Bewertung der EyeLink-Referenzdaten für den Videokalibrierungstest.